# Kotlin 'this' Keyword

## 50 Questions with Answers

This comprehensive guide covers the usage of the **this** keyword in Kotlin with questions ranging from simple to hard difficulty levels. Master the concept through practical examples and coding challenges.

## SIMPLE LEVEL (Questions 1-20)

### 1. What does this refer to in a class?

**Answer:** this refers to the current instance of the class.

### 2. What is the output?

```
class Person(val name: String) { fun display() = println(this.name) } val p = Person("Alice") p.display()
```

**Answer:** Alice

### 3. Is this mandatory when accessing properties?

**Answer:** No, it's optional. You can use 'name' instead of 'this.name'.

### 4. What does this return in a function?

**Answer:** this returns the current object instance.

### 5. Can you use this in a companion object?

**Answer:** No, companion objects don't have instance context.

### 6. What is the output?

```
class Counter { var count = 0 fun increment() { this.count++ } } val c = Counter() c.increment()
println(c.count)
```

**Answer:** 1

### 7. What does this refer to inside an inner class?

**Answer:** The instance of the inner class.

### 8. How do you access outer class this from inner class?

**Answer:** Use this@OuterClassName

### 9. What is the output?

```
class Box { val value = 10 fun show() = this.value } println(Box().show())
```

**Answer:** 10

### 10. Can you use this in a static context?

**Answer:** No, Kotlin doesn't have static. Companion objects don't have this.

### 11. What does this refer to in an extension function?

**Answer:** The receiver object (the object before the dot).

### 12. What is the output?

```
fun String.greet() = "Hello, $this" println("World".greet())
```

**Answer:** Hello, World

### 13. Can you return this from a function?

**Answer:** Yes, often used for method chaining (Builder pattern).

### 14. What is the output?

```
class Builder { var name = "" fun setName(n: String): Builder { this.name = n return this } }
println(Builder().setName("Test").name)
```

**Answer:** Test

### 15. What does this refer to in a lambda with receiver?

**Answer:** The receiver object specified in the lambda type.

### 16. What is the output?

```
class Sample { val x = 5 fun test() = this.x * 2 } println(Sample().test())
```

**Answer:** 10

### 17. Can this be null?

**Answer:** No, this always refers to a valid object instance.

### 18. What is the output?

```
data class User(val id: Int) { fun display() = this.id } println(User(100).display())
```

**Answer:** 100

### 19. Does this work in top-level functions?

**Answer:** No, top-level functions don't have an instance context.

### 20. What is the output?

```
class Test { val value = "Kotlin" fun getValue() = this.value } println(Test().getValue())
```

**Answer:** Kotlin

## MEDIUM LEVEL (Questions 21-35)

### 21. What is the output with nested classes?

```
class Outer { val name = "Outer" inner class Inner { val name = "Inner" fun display() = println(this.name) } } Outer().Inner().display()
```

**Answer:** Inner (this refers to Inner class instance)

### 22. How to access outer class property from inner class?

```
class Outer { val x = 10 inner class Inner { val x = 20 fun sum() = this.x + this@Outer.x } } println(Outer().Inner().sum())
```

**Answer:** 30 (20 from Inner + 10 from Outer)

### 23. What happens with this in apply block?

```
data class Person(var name: String, var age: Int) val p = Person("", 0).apply { this.name = "John" age = 25 // this is optional } println(p)
```

**Answer:** Person(name=John, age=25)

### 24. What is the output with also block?

```
val list = mutableListOf(1, 2, 3).also { println(this) // What does this refer to? }
```

**Answer:** Compilation error! 'this' is not available in 'also' block. Use 'it' instead.

### 25. What is the output using run?

```
class Calculator { var result = 0 fun add(x: Int) = run { this.result += x this } } val c = Calculator().add(5).add(10) println(c.result)
```

**Answer:** 15

### 26. What does this refer to in let?

```
class Box(val value: Int) Box(10).let { println(this) // Error! }
```

**Answer:** Compilation error! 'let' uses 'it' parameter, not 'this'.

### 27. What is the output with multiple this contexts?

```
class Outer { fun outerFun() { val inner = object { fun innerFun() { println(this@Outer) } } inner.innerFun() } }
```

**Answer:** Prints the Outer class instance reference.

### 28. What happens in extension function with this?

```
fun Int.triple() = this * 3 println(5.triple())
```

**Answer:** 15 (this refers to the Int receiver, which is 5)

### 29. What is the output?

```
class Node(val value: Int) { fun next(n: Node) = n.apply { println(this.value) } } Node(1).next(Node(2))
```

**Answer:** 2 (this refers to Node(2) inside apply)

### 30. What does this refer to in with?

```
class Car(val model: String) val car = Car("Tesla") with(car) { println(this.model) }
```

**Answer:** Tesla (this refers to the car object)

### 31. What is the output with constructor this?

```
class User(val name: String) { constructor(name: String, age: Int) : this(name) { println(this.name) } }
User("Alice", 30)
```

**Answer:** Alice

### 32. What happens with lambda receiver?

```
fun StringBuilder.build(block: StringBuilder.() -> Unit) { this.block() } val sb = StringBuilder().build {
this.append("Hello") }
```

**Answer:** StringBuilder with 'Hello' appended

### 33. What is the output?

```
class Counter { private var count = 0 fun increment() = this.also { it.count++ } fun get() = count }
println(Counter().increment().increment().get())
```

**Answer:** 2

### 34. What does this refer to in anonymous object?

```
class Outer { val x = 10 fun create() = object { val y = 20 fun sum() = this.y + this@Outer.x } }
println(Outer().create().sum())
```

**Answer:** 30

### 35. What is the output with scope functions?

```
data class Point(var x: Int, var y: Int) val p = Point(1, 2).apply { x = 10 }.run { y = 20 this }
println(p)
```

**Answer:** Point(x=10, y=20)

## HARD LEVEL (Questions 36-50)

### 36. What is the output with nested scope functions?

```
class Builder { var name = "" var age = 0 fun build() = this.apply { name = "John" }.also { it.age = 30 }.run { "Created: ${this.name}, ${this.age}" } } println(Builder().build())
```

**Answer:** Created: John, 30

### 37. Complex nested class scenario - what is the output?

```
class A { val value = "A" inner class B { val value = "B" inner class C { val value = "C" fun display() = "${this.value}-${this@B.value}-${this@A.value}" } } } println(A().B().C().display())
```

**Answer:** C-B-A

### 38. What happens with extension function on nullable type?

```
fun String?.orDefault() = this ?: "default" println(null.orDefault()) println("test".orDefault())
```

**Answer:** default test (this can be null in nullable extension)

### 39. What is the output with DSL builder?

```
class HTML { private val elements = mutableListOf() fun body(init: BODY.() -> Unit) { elements.add(BODY().apply(init).toString()) } override fun toString() = elements.joinToString() } class BODY { var content = "" override fun toString() = "$content" } val html = HTML().apply { body { this.content = "Hello" } } println(html)
```

**Answer:** Hello

### 40. What is the output with receiver type conflict?

```
class Outer { fun Int.extend() = this + 10 fun test() { println(5.extend()) } } Outer().test()
```

**Answer:** 15 (this refers to Int receiver, which is 5)

### 41. Complex this labeling - what is the output?

```
class A { inner class B { fun A.foo() = this@A fun B.bar() = this@B fun test() { val a = A() println(a.foo() === this@A) println(this.bar() === this@B) } } } A().B().test()
```

**Answer:** true true

### 42. What happens with inline function and this?

```
class Processor { var value = 0 inline fun process(block: Processor.() -> Unit) { this.block() } } val p = Processor().apply { process { value = 100 } } println(p.value)
```

**Answer:** 100

### 43. What is the output with delegation?

```
interface Base { fun print() } class BaseImpl : Base { override fun print() = println(this::class.simpleName) } class Derived : Base by BaseImpl() Derived().print()
```

**Answer:** BaseImpl (this refers to the delegate object)

### 44. Complex scope function chaining - output?

```
data class User(var name: String, var age: Int) val result = User("", 0) .apply { name = "Alice" } .also { println(it.name) } .run { age = 25; this } .let { it.name } println(result)
```

**Answer:** Alice (printed) Alice (final result)

### 45. What is the output with recursive this?

```
class Node(val value: Int, var next: Node? = null) { fun append(value: Int): Node = apply { if (this.next == null) this.next = Node(value) else this.next!!.append(value) } } val node = Node(1).append(2).append(3) println(node.next?.next?.value)
```

**Answer:** 3

### 46. What happens with this in object expression?

```
class Factory { fun create() = object { val data = this@Factory fun getData() = this.data } } val obj = Factory().create() println(obj.getData() is Factory)
```

**Answer:** true

### 47. What is the output with generic extension?

```
fun T.also(block: (T) -> Unit): T { block(this) return this } val result = "Hello".also { println(it.length) } println(result)
```

**Answer:** 5 (printed) Hello (returned)

### 48. Complex builder pattern - what is the output?

```
class Query { private var sql = "" fun select(vararg cols: String) = apply { sql = "SELECT ${cols.joinToString()}" } fun from(table: String) = apply { sql += " FROM $table" } fun build() = this.sql } println(Query().select("id", "name").from("users").build())
```

**Answer:** SELECT id, name FROM users

### 49. What is the output with this in when expression?

```
class Validator(val value: Int) { fun validate() = when { this.value < 0 -> "Negative" this.value == 0 -> "Zero" else -> "Positive" } } println(Validator(-5).validate())
```

**Answer:** Negative

### 50. Ultimate challenge - what is the output?

```
class Outer(val x: Int) { inner class Middle(val y: Int) { inner class Inner(val z: Int) { fun calculate() = buildString { append(this@Inner.z) append(this@Middle.y) append(this@Outer.x) with(this@Outer) { append(this.x) } } } } } println(Outer(1).Middle(2).Inner(3).calculate())
```

**Answer:** 3211

# HARD CODING CHALLENGES

### Challenge 1: Fluent Builder with Validation

Create a UserBuilder class that uses method chaining with 'this'. The builder should: • Have methods: setName(), setEmail(), setAge() that return this • Implement a validate() method that checks all fields are set • Implement a build() method that returns a User object • Use 'this' appropriately throughout

### Challenge 2: Nested Scope Function Challenge

Write a function that processes a list of Person objects using nested scope functions. Requirements: • Use apply to modify each person's properties • Use also to log each modification • Use run to calculate and return statistics • Demonstrate proper use of 'this' vs 'it' in each scope

### Challenge 3: DSL for HTML Generation

Create a type-safe DSL for generating HTML using lambda with receivers: • Create classes: HTML, HEAD, BODY, DIV, P • Each should support nested elements using 'this' as receiver • Support attributes and text content • Example usage: html { head { title { +"My Page" } }; body { div { +"Content" } } }

### Challenge 4: Complex Inner Class Navigation

Create a file system simulator with nested classes: • FileSystem (outer) with property rootPath • Directory (inner) with property name • File (inner of Directory) with property fileName • File should have a method getFullPath() that uses this@FileSystem, this@Directory, and this • Demonstrate accessing all three levels of 'this'

### Challenge 5: Generic Extension with This

Create a generic extension function framework: • Write a generic extension function T.validate(validator: T.() -> Boolean) • It should use 'this' to refer to the receiver • Create validators for String, Int, and custom classes • Chain multiple validators using 'this' • Return ValidationResult with details

## Tips for Mastering 'this' in Kotlin

**1. Labeled this:** Use this@ClassName to access outer class instances from inner classes. **2. Scope Functions:** • apply, run, with → use 'this' as receiver • let, also → use 'it' as parameter **3. Extension Functions:** 'this' refers to the receiver object. **4. Lambda with Receiver:** 'this' refers to the implicit receiver type. **5. Method Chaining:** Return 'this' from methods for fluent APIs. **6. Inner Classes:** Inner classes can access outer class 'this' using labeled this. **Practice regularly and experiment with different contexts to master the concept!**